Rheinisch-Westfälische Technische Hochschule Aachen

Bachelor Thesis

A Very Nice Bachelor Topic

by
Jan-Christoph Kassing

Lehr- und Forschungsgebiet Informatik 2
Programming Languages and Verification

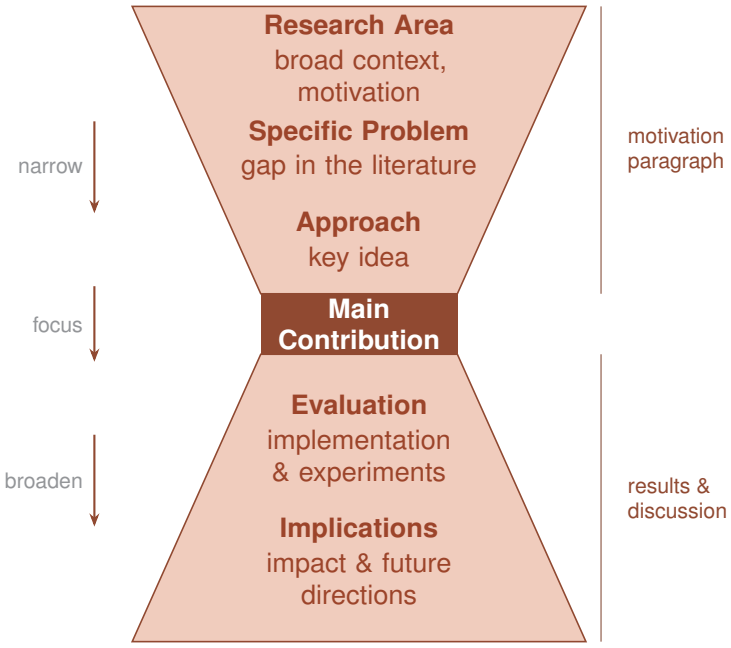
First Examiner:
[Prof. Dr. Name of First Examiner]

Second Examiner:
[Prof. Dr. Name of Second Examiner]

Supervisor: [Name of Supervisor]

October 27, 2026

Abstract



Hourglass structure of a good abstract. Start broad (research area and problem), funnel down to the specific contribution, then widen again to state results and implications.

Write a self-contained summary of roughly 150–250 words following the hourglass structure above. Cover the following six points in order:

1. **Research area.** Name the field (e.g., automated termination analysis, probabilistic verification, complexity analysis of rewrite systems) and state in one sentence what the field studies and why it matters.

2. **Specific problem.** State precisely which problem this thesis addresses and why it is challenging or not yet fully solved by existing approaches.

3. **Approach.** Describe the key idea of your contribution in one or two sentences. Avoid mathematical notation and abbreviations, and use informal language.

4. **Theoretical results.** Name the main property proved (soundness, completeness, an equivalence result, or a complexity bound) and state it informally.

5. **Implementation and evaluation (if applicable).** Name the tool and the benchmark set (e.g., the Termination Problems Data Base) and give the headline result: how many more examples are solved compared to the baseline and what is the runtime overhead.

6. **Implications.** Explain how your results solve the issue or fill the gap you pointed out beforehand.

Acknowledgements

I want to express my most profound appreciation to my supervisor!

Maybe I want to thank someone else as well...

Contents

Abstract **iii**

Acknowledgements **v**

Contents **vi**

1 Introduction **1**

2 Preliminaries **3**
2.1 Basic Definitions . . . 3
2.2 Known Results from the Literature . . . 5

3 [Title of Your Main Contribution] **7**
3.1 [Central Definitions / The New Framework] . . . 7
3.2 [Main Results] . . . 7

4 Evaluation **9**
4.1 Implementation . . . 9
4.2 Experimental Setup . . . 9
4.3 Results . . . 10
4.4 Discussion . . . 10

5 Conclusion **11**

Bibliography **12**

Bibliography Style **14**
Authors . . . 14
Titles . . . 14
Venues . . . 15
DOI . . . 17
Non-English Sources . . . 17
Results . . . 17
Ordering . . . 18

Omitted Proofs **19**
Proof of [Name of Main Theorem] . . . 19

Mathematical Writing **20**
Structure and Flow . . . 20
Definitions and Notation . . . 20
Formulas in the Text . . . 21
Theorems and Proofs . . . 22

Language and Style 23
Figures and Tables 23

CHAPTER 1

Introduction

The introduction motivates the topic and provides an overview of the thesis. It can be seen as a longer version of the abstract that follows the hourglass structure again, but with emphasis on the top part of the hourglass. However, you can now use citations and explain everything in more detail. It should be accessible to any computer scientist without prior knowledge of the specific area.

The background and motivation part should cover the following points:

1. Name the research area and describe informally what objects are studied (e.g., terms, integer programs, Java programs, automata) and what property is of interest (e.g., termination, complexity, safety).
2. Explain why the property matters.
3. Give a small, self-contained example that illustrates the problem informally.
4. Explain why answering the question automatically is non-trivial and what makes the problem hard in general.

The open-problem part should contain:

1. A summary of what is already known in the area (prior results, existing automated techniques, tool support).
2. A precise identification of the gap: e.g., no automated procedure exists, existing techniques are unsound or incomplete for a relevant subclass, or no efficient algorithm is known.
3. A clear statement of what this thesis aims to achieve.

Related Work

Discuss the most relevant prior work. For each cited work, address the following:

1. What does the work do? Describe the approach in one to three sentences and do not just list references.
2. How does it relate to this thesis? Does it solve a special case, use a different technique, or target a different formalism?
3. What limitation motivates your own contribution?

Contribution

List one item per concrete contribution. Each item should:

1. Identify what is new (a technique, a criterion, a framework, an implementation, or an experimental evaluation).
2. State the property proved, if applicable (soundness, completeness, both, or a complexity bound).
3. Reference the theorem, or section in which the contribution appears.

Structure

End the introduction by outlining the remaining chapters. Give at least one sentence per chapter describing its content. Do not simply write "Chapter X presents the main results."

Example: "The remainder of this thesis is organized as follows. Chapter 2 introduces the formal background required throughout this thesis, including ... Then, Chapter 3 presents ... Chapter 4 describes the implementation and the experimental evaluation. Finally, Chapter 5 summarizes the results and discusses directions for future work."

CHAPTER 2

Preliminaries

The opening paragraph of a chapter should briefly outline the chapter structure by naming each section and stating what it covers.

In the preliminaries you can name the formalism studied (e.g., term rewriting systems, integer transition systems, probabilistic programs) and cite a standard reference for further reading, e.g., [4]. Do not add a citation to every definition. Format all entries of your bibliography as described in Appendix , and follow the guidelines for mathematical writing in Appendix .

2.1 Basic Definitions

Introduce the formal objects of the theory bottom-up: start with the most elementary notion (e.g., the syntactic objects) and build towards the property of interest (e.g., termination, complexity).

Each definition in this section should be accompanied by:

1. A sentence stating what the definition formalizes and why it is needed.
2. The formal statement in a Definition box.
3. A brief explanation of each component of the definition.
4. At least one example.

Cover the following in order:

1. **Syntactic Objects.** Define the elementary building blocks of the formalism (e.g., the signature and the set of terms, the program syntax, the set of states and transitions).
2. **Evaluation Relation.** Define how one object evaluates to another. If several strategies exist, introduce them here and state which one is used in the rest of the thesis.
3. **Derived Notions.** Define derived concepts such as normal forms, reachability, derivation lengths, etc.
4. **The Property of Interest.** Give the formal definition of the property the thesis analyzes (e.g., termination, innermost complexity, almost-sure termination).

In the beginning, we start by defining [Basic Notion] that is used throughout the remainder of this thesis.

Definition 2.1.1: [Name of First Basic Notion]

A [Thing] is [Basic Notion] if it satisfies [Property 1] and [Property 2].

The reason for requiring [Property 1] is [...]. [Property 2] is generally assumed in the literature.

Example 2.1.2.

Example 2.1.3.

How to Reference Definitions, Theorems, and Examples

Never write numbers like "Definition 2.1.1" by hand. Instead, refer to the label with `\cref`, which prints the type and the number and turns both into a clickable link. For example, `\cref{def:first-notion}` yields Definition 2.1.1, and `\cref{ex:first-notion}` yields Example 2.1.2. How the label is set depends on the environment:

- **Boxes** (`Definition`, `Theorem`, `Lemma`, `Corollary`) take the label as their second argument, *without* prefix. The box adds the prefix itself, so
  `\begin{Definition}{Title}{first-notion}`
  creates the label `def:first-notion`.
- **All other environments** (`example`, `remark`, `notation`, `equation`, `table`, ...) and sectioning commands get an explicit `\label`, and you choose the prefix yourself, e.g., `\begin{example}\label{ex:first-notion}`. For tables and figures, put the `\label` after the `\caption`.

Table 2.1: How to label and reference the most common environments.

You write	Reference with	Output
`\begin{Definition}{...}{first-notion}`	`\cref{def:first-notion}`	Definition 2.1.1
`\begin{Lemma}{...}{aux-lemma}`	`\cref{lem:aux-lemma}`	Lemma 2.2.1
`\begin{Theorem}{...}{key-theorem}`	`\cref{thm:key-theorem}`	Theorem 2.2.2
`\begin{Corollary}{...}{key-corollary}`	`\cref{cor:key-corollary}`	Corollary 2.2.4
`\begin{example}\label{ex:first-notion}`	`\cref{ex:first-notion}`	Example 2.1.2
`\section{...}\label{sec:known-results}`	`\cref{sec:known-results}`	Section 2.2
`\chapter{...}\label{chap:main-chapter}`	`\cref{chap:main-chapter}`	Chapter 3
`\caption{...}\label{tab:results}`	`\cref{tab:results}`	Table 4.1

Further useful variants:

- Several labels at once:
  `\cref{ex:first-notion,ex:first-notion-second-example}` yields "Examples 2.1.2 and 2.1.3".
- Only the number, e.g., inside a sentence that already names the type:
  `\ref{def:first-notion}` yields "2.1.1".
- The page: `\cpageref{thm:key-theorem}` yields "Page 5".

2.2 Known Results from the Literature

Present the lemmas and theorems from the literature that are used in Chapter 3. You do not have to prove these results unless the proof is insightful for the rest of the thesis. Always cite the original source. Each result in this section should be presented as follows:

1. A sentence explaining why the result is important and how it will be used in the thesis.
2. The formal statement in a Lemma or Theorem box, with a citation in the title.
3. A proof indication: either a short proof, "Proof: see [**?**]", or a forward reference to the appendix (see Appendix).
4. (Optional) A brief remark on the scope or limitations of the result if that is relevant for understanding its use in Chapter 3.

This section collects known results from the literature that are used in the rest of the thesis. Organize them as follows:

1. **Auxiliary lemmas.** State small technical results about the basic objects from Section 2.1 (e.g., properties of substitution, closure conditions, or monotonicity). These are typically needed in the proofs of Chapter 3.
2. **Characterization theorem(s).** State the main theoretical results from the literature about the property of interest. These serve as a reference point for understanding what is already known.
3. **Existing automated technique(s).** If the thesis extends an existing automated method (e.g., dependency pairs, polynomial interpretations, ranking functions), state its correctness theorem here.

Lemma 2.2.1: [Name of Auxiliary Lemma], [7]

A Lemma is a small result or auxiliary result, which is only interesting because we can use it to prove the main result.

Proof. The complete proof can be found in [7]. □

Theorem 2.2.2: [Name of Key Theorem], [7]

A Theorem is one of the main results. Everything else is defined and proven such that this results can be proven.

Proof. The complete proof can be found in [7]. □

Example 2.2.3. Apply Theorem 2.2.2 to a concrete example. Its proof relies on Lemma 2.2.1. Demonstrate both a case where the result applies successfully and, if possible, a case where it fails due to the restriction identified above. The failing case motivates the extension in Chapter 3.

Corollary 2.2.4: [Name of a Direct Consequence], [7]

A Corollary is a (easy) consequence of some other (main) result. Still, it has to be proven, even if the proof is rather simple!

Proof. The complete proof can be found in [7]. □

Example 2.2.5. Example for Corollary 2.2.4.

You can use colors to highlight differences in boxes or even difficult parts in proofs. However, do not make the colors too distracting and everything has to be readable in black and white as well.

CHAPTER 3

[Title of Your Main Contribution]

This chapter contains the main theoretical (and possibly technical) contribution of the thesis. It should present a coherent body of new results, organized into sections.

For a theory-focused thesis: introduce new notions, state lemmas/theorems, provide proofs, and illustrate results with examples. For an implementation-focused thesis: additionally describe the algorithm, key design decisions, and data structures.

The opening paragraph of this chapter should cover the following points:

1. State what new objects, criteria, or techniques this chapter introduces.
2. Introduce a running example that will be used throughout to illustrate each new definition and result.
3. Outline the chapter structure by referencing the main sections.

3.1 [Central Definitions / The New Framework]

For self-introduced notions it is even more important to explain everything and use multiple examples to illustrate everything. Use one running example that can be used through the whole section and maybe some additional small examples to highlight special cases.

Definition 3.1.1: [Name of Your First Definition]

Example 3.1.2.

3.2 [Main Results]

Each result in this section should be presented as follows:

1. State and prove all auxiliary lemmas before the main theorem.
2. State the main theorem. Typical statements are:
   - **Soundness:** If the technique returns YES, then the property holds.
   - **Completeness:** If the property holds, the technique returns YES.
   - **Equivalence:** The new criterion holds if and only if an existing criterion holds.

3. Provide a complete proof, structured into numbered cases for long arguments. Reference definitions and earlier results by label. If a proof is long or very technical, give a proof sketch here and move the full proof to Appendix (see Theorem 3.2.2 for an example).

4. Apply the theorem to the running example from Example 3.1.2, walking through every step explicitly.

Lemma 3.2.1: [Name of Auxiliary Lemma]

Proof. □

Theorem 3.2.2: [Name of Main Theorem, e.g., Soundness]

Proof Sketch. [Main idea of the proof in a few sentences.] The full proof can be found in Appendix . □

Example 3.2.3. Apply Theorem 3.2.2 to the running example from Example 3.1.2 via Lemma 3.2.1. Walk through every step of the argument explicitly.

CHAPTER 4

Evaluation

This chapter demonstrates that the approach from Chapter 3 works in practice.

A good evaluation chapter:

- Describes clearly what was implemented and where.
- Explains the experimental setup with enough detail to be reproducible.
- Presents results honestly, including cases where the technique does not help.
- Interprets the numbers and connects them back to the theory.

4.1 Implementation

The implementation section should cover:

1. The name of the tool extended and a brief description of its overall architecture (e.g., the pipeline from input parsing to proof output) and any external solvers used.
2. Which module(s) you added or changed and how they fit into the existing architecture.

4.2 Experimental Setup

The experimental setup should specify:

1. **Benchmark set.** Which set was used (e.g., TPDB, TermComp, SV-COMP, or a custom set)? How many examples total? Are there relevant subsets by category, strategy, or problem type?
2. **Configurations compared.** At minimum include the unmodified baseline and the tool with your extension. List any competing tools with a brief description.
3. **Hardware and time limits.** CPU model, amount of RAM, operating system, and per-example wall-clock timeout.

4.3 Results

The results section should:

1. Present results in a table (see Table 4.1).
2. Summarize the headline finding directly below the table, citing concrete numbers (additional YES answers, percentage improvement, overhead in seconds).
3. Point out interesting subsets or individual examples where the technique is especially effective and where the technique is not effective.
4. Note any surprising results to be discussed in Section 4.4.

Table 4.1 summarizes the results of our experiments.

Table 4.1: Experimental results on [benchmark set name]. *YES*/*NO* denotes the number of examples for which [the property / its negation] was proved. *MAYBE* means the tool returned no answer. *TO* means the timeout was exceeded. *Avg. Time* is the average wall-clock time per example in seconds.

Configuration	**YES**	**NO**	**MAYBE**	**TO**	**Avg. Time (s)**
[Tool without extension]	–	–	–	–	–
[Tool with extension]	–	–	–	–	–

4.4 Discussion

The discussion should address the following questions:

1. Do the results match theoretical expectations from Chapter 3? If not, explain why.
2. On which types of examples does the technique help most? Identify structural properties of the benchmark examples that explain the gain.
3. Are there cases where the technique does not help or is slower? Explain the reason and relate it to the theoretical properties.
4. What are the main limitations and how could they be overcome? (This can bridge to the future work discussion in Chapter 5.)

CHAPTER 5

Conclusion

The conclusion summarizes the thesis and discusses future work. Do not introduce any new material here. A reader who reads only the introduction and the conclusion should get a coherent picture of what was done and what remains open. It should follow the same structure as the introduction and the abstract, but with more emphasis on the bottom part of the hourglass.

The summary should address the following points:

1. Restate the problem studied and explain why it is relevant.
2. Describe the main contribution and the approach taken (reference Chapter 3).
3. Name the central theoretical result (reference Theorem 3.2.2) and state informally what it guarantees.
4. Summarize the experimental findings – number of additional examples solved, runtime overhead – if an evaluation was conducted (reference Chapter 4).

Each future work item should:

1. Identify a specific limitation of the current approach or a natural extension that was beyond the scope of this thesis.
2. Describe what would be needed to address it (new theoretical machinery, a different encoding, a richer formalism, etc.).
3. Explain what benefit this would bring (more examples solved, a stronger guarantee, applicability to a wider class of systems).

Bibliography

[1] Emma Ahrens, Jan-Christoph Kassing, Jürgen Giesl, and Joost-Pieter Katoen. Weighted Rewriting: Semiring Semantics for Abstract Reduction Systems. In *Proc. FSCD '25*, LIPIcs 337, pages 6:1–6:21, 2025. doi:10.4230/LIPIcs.FSCD.2025.6.

[2] Website of AProVE: https://aprove.informatik.rwth-aachen.de/. Accessed: 2026-07-19.

[3] Thomas Arts and Jürgen Giesl. Termination of Term Rewriting Using Dependency Pairs. *Theor. Comput. Sci.*, 236:133–178, 2000. doi:10.1016/S0304-3975(99)00207-8.

[4] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998. doi:10.1017/CBO9781139172752.

[5] Marc Brockschmidt, Carsten Otto, Christian von Essen, and Jürgen Giesl. Termination Graphs for Java Bytecode. In Simon Siegler and Nathan Wasser, editors, *Verification, Induction, Termination Analysis*, LNCS 6463, pages 17–37. Springer, 2010. doi:10.1007/978-3-642-17172-7_2.

[6] Carsten Fuhs, Jürgen Giesl, Michael Parting, Peter Schneider-Kamp, and Stephan Swiderski. Proving Termination by Dependency Pairs and Inductive Theorem Proving. *J. Autom. Reason.*, 47:133–160, 2011. doi:10.1007/s10817-010-9215-9.

[7] Jürgen Giesl, René Thiemann, Peter Schneider-Kamp, and Stephan Falke. Mechanizing and Improving Dependency Pairs. *J. Autom. Reason.*, 37:155–203, 2006. doi:10.1007/s10817-006-9057-7.

[8] George D. Gopen and Judith A. Swan. The Science of Scientific Writing. *Am. Sci.*, 78(6):550–558, 1990.

[9] Paul R. Halmos. How to Write Mathematics. *Enseign. Math.*, 16:123–152, 1970.

[10] Nicholas J. Higham. *Handbook of Writing for the Mathematical Sciences*. SIAM, third edition, 2020. doi:10.1137/1.9781611976106.

[11] Jan-Christoph Kassing. Using Dependency Pairs for Proving Almost-Sure Termination of Probabilistic Term Rewriting. Master's thesis, RWTH Aachen University, 2022.

[12] Jan-Christoph Kassing and Jürgen Giesl. Proving Almost-Sure Innermost Termination of Probabilistic Term Rewriting Using Dependency Pairs. In *Proc. CADE '23*, LNCS 14132, pages 344–364, 2023. doi:10.1007/978-3-031-38499-8_20.

[13] Jan-Christoph Kassing and Jürgen Giesl. From Innermost to Full Probabilistic Term Rewriting: Almost-Sure Termination, Complexity, and Modularity. *Log. Methods Comput. Sci.*, 21(4), 2025. doi:10.46298/lmcs-21(4:28)2025.

[14] Donald E. Knuth, Tracy Larrabee, and Paul M. Roberts. *Mathematical Writing*. MAA Notes 14. Mathematical Association of America, 1989.

[15] Leslie Lamport. How to Write a 21st Century Proof. *J. Fixed Point Theory Appl.*, 11:43–63, 2012. doi:10.1007/s11784-012-0071-6.

[16] Nils Lommen, Éléanore Meyer, and Jürgen Giesl. Control-Flow Refinement for Complexity Analysis of Probabilistic Programs in KoAT (Short Paper). In *Proc. IJCAR '24*, LNCS 14739, pages 233–243, 2024. doi:10.1007/978-3-031-63498-7_14.

[17] René Thiemann. *The DP Framework for Proving Termination of Term Rewriting*. PhD thesis, RWTH Aachen University, 2007.

[18] Justin Zobel. *Writing for Computer Science*. Springer, third edition, 2014. doi:10.1007/978-1-4471-6639-9.

Bibliography Style

The rules below explain how to properly format the bibliography of your thesis. This template uses the standard BibTeX style `plainurl`, a variant of `plain` that prints DOIs as links (see `\bibliographystyle` in `main.tex`). All entries of this chapter are contained in `bibliography.bib` and are cited here, so you can compare the source of each entry with the resulting output in the bibliography at the end of this thesis.

Authors

- Full first names, all authors, never "et al." inside the entry:
  `author = {Lommen, Nils and Meyer, {\'E}l{\'e}anore and Giesl, J{\"u}rgen}`
- Accents in LaTeX form:
  `Giesl, J{\"u}rgen`, `Meyer, {\'E}l{\'e}anore`, `Thiemann, Ren{\'e}`.
- Name particles lowercase and braced, so they stay with the surname:
  `{von Essen}, Christian`.
- Hyphenated surnames braced: `{Schneider-Kamp}, Peter`.

Titles

- Title Case for every main word, protected with braces:

```
{{Proving}} {{Almost-Sure}} {{Innermost}} {{Termination}} of
{{Probabilistic}} {{Term}} {{Rewriting}} {{Using}} {{Dependency}} {{Pairs}}
```

- Tool names in sans-serif via the macro `\tool`, which is defined in `verifyThesisStyle.sty`, so that every tool name looks the same in the text and in the bibliography: `{{\tool{AProVE}}}` yields AProVE. The double braces protect the macro from BibTeX's case changes:

```
{{\tool{KoAT}}}: {{Automatic Complexity}} and {{Termination Analysis}}
of {{Integer Programs}}
```

- Mathematical symbols in titles as math: `Non-$E$-Overlapping`.

Venues

Conferences. Use one fixed pattern, always the same for the same series: `Proc.\ ACRONYM~'YY`, then series with volume, pages, and year. No "Proceedings of the 30th International Conference on . . . ", no ordinal, no city, no publisher. The following two entries are rendered as [12] and [1].

```
@inproceedings{kassinggiesl2023iast,
  author    = {Kassing, Jan-Christoph and Giesl, J{\"u}rgen},
  title     = {{{Proving}} {{Almost-Sure}} {{Innermost}} {{Termination}} of
               {{Probabilistic}} {{Term}} {{Rewriting}} {{Using}}
               {{Dependency}} {{Pairs}}},
  booktitle = {Proc.\ CADE~'23},
  series    = {LNCS 14132},
  pages     = {344--364},
  year      = 2023,
  doi       = {10.1007/978-3-031-38499-8_20}
}

@inproceedings{ahrens2025WeightedRewritingSemiring,
  author    = {Ahrens, Emma and Kassing, Jan-Christoph and Giesl, J{\"u}rgen
               and Katoen, Joost-Pieter},
  title     = {{{Weighted}} {{Rewriting}}: {{Semiring}} {{Semantics}} for
               {{Abstract}} {{Reduction}} {{Systems}}},
  booktitle = {Proc.\ FSCD~'25},
  series    = {LIPIcs 337},
  pages     = {6:1--6:21},
  year      = 2025,
  doi       = {10.4230/LIPIcs.FSCD.2025.6}
}
```

Further venue examples: `Proc.\ IJCAR~'24` (LNCS 14739), `Proc.\ TACAS~'26` (LNCS 16506), `Proc.\ CAV~'26` (LNCS 16684), `Proc.\ FroCoS~'05` (LNCS 3717), `Proc.\ RTA~'10` (LIPIcs 6). LNAI volumes are written as LNCS. Joint events keep the joint acronym (`Proc.\ RTA-TLCA~'14`). Workshops without a series get only pages and year (`Proc.\ WST~'23`, `Proc.\ IWC~'12`).

Journals. Use the ISO abbreviation, volume, pages, and year. Add the issue number only when the pages do not identify the paper (LMCS, article-numbered journals). The following entries are rendered as [6], [3], and [13].

```
@article{fuhs2011provingterminationdependency,
  author  = {Fuhs, Carsten and Giesl, J{\"u}rgen and Parting, Michael and
             {Schneider-Kamp}, Peter and Swiderski, Stephan},
  title   = {{{Proving}} {{Termination}} by {{Dependency}} {{Pairs}} and
             {{Inductive}} {{Theorem}} {{Proving}}},
  journal = {J.\ Autom.\ Reason.},
  volume  = 47,
  pages   = {133--160},
  year    = 2011,
  doi     = {10.1007/s10817-010-9215-9}
}

@article{arts2000termination,
  author  = {Arts, Thomas and Giesl, J{\"u}rgen},
  title   = {{{Termination}} of {{Term}} {{Rewriting}} {{Using}}
             {{Dependency}} {{Pairs}}},
  journal = {Theor.\ Comput.\ Sci.},
  volume  = 236,
```

```
  pages   = {133--178},
  year    = 2000,
  doi     = {10.1016/S0304-3975(99)00207-8}
}

@article{kassing2025LMCS,
  author  = {Kassing, Jan-Christoph and Giesl, J{\"u}rgen},
  title   = {{{From}} {{Innermost}} to {{Full}} {{Probabilistic}} {{Term}}
             {{Rewriting}}: {{Almost-Sure}} {{Termination}}, {{Complexity}},
             and {{Modularity}}},
  journal = {Log.\ Methods Comput.\ Sci.},
  volume  = 21,
  number  = 4,
  year    = 2025,
  doi     = {10.46298/lmcs-21(4:28)2025}
}
```

Other journal abbreviations:

```
J.\ ACM                          Inf.\ Comput.
J.\ Symb.\ Comput.               Inf.\ Process.\ Lett.
ACM Trans.\ Comput.\ Log.        J.\ Log.\ Algebr.\ Methods Program.
```

Book chapters. Use `@incollection` with `booktitle`, `editor`, `publisher`, series, and pages. The following entry is rendered as [5].

```
@incollection{brockschmidt2010TerminationGraphsJava,
  author    = {Brockschmidt, Marc and Otto, Carsten and {von Essen}, Christian
               and Giesl, J{\"u}rgen},
  title     = {{{Termination}} {{Graphs}} for {{\tool{Java}}} {{Bytecode}}},
  booktitle = {Verification, {{Induction}}, {{Termination Analysis}}},
  editor    = {Siegler, Simon and Wasser, Nathan},
  publisher = {Springer},
  series    = {LNCS 6463},
  pages     = {17--37},
  year      = 2010,
  doi       = {10.1007/978-3-642-17172-7_2}
}
```

Theses. Use `@phdthesis` or `@mastersthesis` with the university in `school`. The following entries are rendered as [17] and [11].

```
@phdthesis{thiemann_diss:2007,
  author = {Thiemann, Ren{\'e}},
  title  = {{{The}} {{DP}} {{Framework}} for {{Proving}} {{Termination}} of
            {{Term}} {{Rewriting}}},
  school = {RWTH Aachen University},
  year   = 2007
}

@mastersthesis{kassing:thesis:2022,
  author = {Kassing, Jan-Christoph},
  title  = {{{Using}} {{Dependency}} {{Pairs}} for {{Proving}} {{Almost-Sure}}
            {{Termination}} of {{Probabilistic}} {{Term}} {{Rewriting}}},
  school = {RWTH Aachen University},
  year   = 2022
}
```

A bachelor's thesis is entered as `@mastersthesis` with `type = {Bachelor's thesis}`.

Technical reports. Use `@techreport` with `number = {Technical Report ...}` and `institution = {...}`.

Online material. Use `@misc` with the URL and access date in `note`. The following entry is rendered as [2].

```
@misc{aprove-website,
  key  = {{\relax APrW}},
  note = {Website of \tool{AProVE}:
          \url{https://aprove.informatik.rwth-aachen.de/}.
          Accessed: 2026-07-19}
}
```

DOI

Every entry that has a DOI gets `doi = {10....}`. The style renders it as a link. Workshop papers without a DOI get the proceedings URL in `note`. Never put a URL into the `doi` field.

Non-English Sources

Add `note = {In Japanese}` (or the respective language) for papers in non-English journals.

Results

With these fields the style produces, uniformly:

> Jan-Christoph Kassing and Jürgen Giesl. Proving Almost-Sure Innermost Termination of Probabilistic Term Rewriting Using Dependency Pairs. In *Proc. CADE '23*, LNCS 14132, pages 344–364, 2023. doi:10.1007/978-3-031-38499-8_20.

> Carsten Fuhs, Jürgen Giesl, Michael Parting, Peter Schneider-Kamp, and Stephan Swiderski. Proving Termination by Dependency Pairs and Inductive Theorem Proving. *J. Autom. Reason.*, 47:133–160, 2011. doi:10.1007/s10817-010-9215-9.

> Nils Lommen, Éléanore Meyer, and Jürgen Giesl. Control-Flow Refinement for Complexity Analysis of Probabilistic Programs in KoAT (Short Paper). In *Proc. IJCAR '24*, LNCS 14739, pages 233–243, 2024. doi:10.1007/978-3-031-63498-7_14.

These are the entries [12], [6], and [16] in the bibliography of this thesis.

Ordering

The bibliography is sorted alphabetically by author, not by order of citation: first by the surname of the first author, then by the following authors, then by year, then by title. Entries without authors (websites, tools) sort under their `key` field, which also determines their label (see the `@misc` example in Appendix). Particles are part of the surname for sorting, so `{von Essen}, Christian` is filed under V (between Thiemann and Wasser), not under E. For the examples above this gives the order

> Arts, Giesl (2000) – Brockschmidt, Otto, von Essen, Giesl (2010) – Fuhs, Giesl, Parting, Schneider-Kamp, Swiderski (2011) – Kassing, Giesl (2023) – Kassing, Giesl (2025) – Lommen, Meyer, Giesl (2024) – Thiemann (2007).

Keep the `.bib` file itself in the same order (e.g., with `bibtool -s`, sort key first author and year), so that inconsistencies between entries of the same author are easy to spot.

Omitted Proofs

This appendix contains the full proofs that were omitted from the main text. Move a proof here if it is long or very technical and would interrupt the flow of the main text. Short proofs and proofs that contain an important idea stay in the main text.

For every proof that you move here, follow this pattern:

1. In the main text, directly after the statement, give a proof sketch that explains the main idea in a few sentences and refers to the full proof, e.g., "The full proof can be found in Appendix ." See the proof sketch of Theorem 3.2.2 for an example.
2. Here, start a new section for each proof and begin the proof by referring back to the statement it proves, e.g., "Proof of Theorem 3.2.2", so that the reader can jump back to it.
3. Keep the order of the main text.

Proof of [Name of Main Theorem]

Proof of Theorem 3.2.2. Recall the statement of Theorem 3.2.2: [informal restatement of the theorem].

[Full proof, structured into numbered cases for long arguments. Reference definitions and earlier results by label, e.g., Definition 3.1.1 and Lemma 3.2.1.] □

Mathematical Writing

The guidelines below summarize how to write a clear mathematical thesis. They are taken from classic sources on mathematical and scientific writing, which are worth reading in full: Halmos [9], Knuth et al. [14], Gopen and Swan [8], Lamport [15], Zobel [18], and Higham [10]. The guiding principle behind all of them is that you write for the *reader*: a fellow student who knows the basics of computer science but not your topic. Everything that makes the reader stop, search, or guess is a problem of the text, not of the reader.

Structure and Flow

- **Intuition first, then formality, then an example.** Before each definition or theorem, say in words what it formalizes and why it is needed. After it, illustrate it with an example. Use one running example throughout a chapter instead of many unrelated ones.
- **One topic per paragraph.** The first sentence of a paragraph states its topic, the remaining sentences develop it.
- **Old information first, new information last.** Start a sentence with what the reader already knows and put the new or important part at the end, where the reader expects the emphasis. Keep the subject and its verb close together.
- **Signposting.** Begin every chapter by saying what it contains and end it by summarizing what was achieved and where it is used next. Within long sections, tell the reader where the argument is going ("To prove Theorem 3.2.2, we first show two auxiliary lemmas.").
- **Present only what you need.** The preliminaries contain exactly the notions used later. Do not include definitions or results "for completeness" that play no role in your thesis.

Definitions and Notation

- **Define everything before you use it,** and define it only once. If a notion was defined many pages before, recall it briefly and refer to the definition ("Recall that a term is *linear* if ... (see Definition 2.1.1).").
- **Emphasize the defined term** where it is defined, e.g., "A relation is called *terminating* if ... ".

- **Definitions use "if" and "=".** In a definition, "if" already means "if and only if", so never write "iff" or "if and only if" in a definition. Likewise, define objects with "=" ("Let $f(x) = x + 1$."), not with `:=`, `\coloneqq`, or `::=`. Use "iff" or "if and only if" only in statements that you prove, e.g., in a theorem or lemma, where both directions have to be shown.
- **One symbol, one meaning.** Never use the same symbol for two different objects, and never use two symbols for the same object. Keep the conventions consistent: e.g., lowercase letters for elements, uppercase letters for sets, calligraphic letters for systems.
- **Do not introduce notation that is used only once.** Every new symbol is a burden for the reader; a few words are often clearer.
- **One word, one concept.** In technical writing, varying the words for the same concept (term, expression, object) suggests that they mean different things. Pick one word and stick to it.
- **Use standard notation** wherever it exists, e.g., the notation of the standard textbook of your area. If your thesis uses many symbols, add a list of notation to the front matter.

Formulas in the Text

Mathematical formulas are part of the sentences they appear in and follow the same rules of grammar and punctuation.

- **Do not start a sentence with a symbol.**
- **Do not separate two formulas by only a punctuation mark**; put words in between.
- **Do not use logical symbols as abbreviations in prose.** Symbols like $\forall$, $\exists$, $\Rightarrow$, or $\wedge$ belong inside formulas, not between words.
- **Punctuate displayed formulas.** A displayed formula ends with a comma or a period if the sentence does, e.g.,

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1, \tag{1}$$

  where $n \in \mathbb{N}$.
- **Numbering.** Use `\begin{equation}` with a `\label` for displayed formulas that you refer to, and refer to them with `\cref`, e.g., `\cref{eq:mw-geometric-sum}` yields Equation (1). Use `\begin{equation*}` for formulas that are never referenced.
- **Words in formulas** are set with `\text`, and operator names with `\operatorname` or the predefined commands such as `\max` and `\log`, so that they are not typeset as products of variables: $\max(x, y)$ instead of $max(x, y)$.

- **Use the right symbols:** `\colon` for the type of a function ($f\colon \mathbb{N} \to \mathbb{N}$), `\mid` in set comprehensions ($\{x \in \mathbb{N} \mid x > 0\}$), `\ldots` between commas ($x_1, \ldots, x_n$), and `\cdots` between operators ($x_1 + \cdots + x_n$).

The following table shows typical mistakes and how to avoid them.

Table 1: Typical mistakes in mathematical prose.

Avoid	**Prefer**
x is positive, so …	The value x is positive, so …
Since $x > 0$, $y < 0$.	Since $x > 0$, we have $y < 0$.
$\forall x \in S$ we have $x > 0$.	For all $x \in S$, we have $x > 0$.
If n is even $\Rightarrow n^2$ is even.	If n is even, then n^2 is even.
As shown in Theorem 2.2.2, …	As shown in Theorem 2.2.2, …
This follows from it.	This lemma follows from Theorem 2.2.2.
Obviously, the relation is terminating.	The relation is terminating, since every rule decreases the size of the term.
(Definition) A term is *linear* iff …	(Definition) A term is *linear* if …
(Definition) Let $f(x) \coloneqq x + 1$.	(Definition) Let $f(x) = x + 1$.

Theorems and Proofs

- **Self-contained statements.** A theorem lists all of its assumptions. Avoid statements like "Under the assumptions above, …", since the reader has to search for what "above" means.
- **Explain the significance** of every result: why it is needed, how it is used later, and what it means intuitively.
- **Announce the structure of a proof** at its beginning ("We prove the claim by induction on n.", "Assume for a contradiction that …"). Mark every case clearly and state the induction hypothesis explicitly.
- **Justify every step** and refer to the definition or result it uses by its label. Structure long proofs hierarchically into numbered steps, each with its own short justification.
- **Avoid "obviously", "clearly", and "trivially".** If a step is obvious, these words are unnecessary; if it is not, they hide a gap. Give the reason instead.
- **Long proofs go to the appendix.** Give a proof sketch in the main text and move the full proof to Appendix .
- **Apply the result to an example** after the proof, ideally to the running example.

The following lemma and its proof illustrate these guidelines. The lemma explains its significance before the statement, the proof announces its structure, marks both cases, states the induction hypothesis, and justifies the only non-trivial step.

We now show that the sum of the first n odd numbers is a square number. This allows us to compute such sums without adding up all summands.

Lemma .0.1: Sum of Odd Numbers

For all $n \in \mathbb{N}$, we have

$$\sum_{i=1}^{n}(2i-1) = n^2. \tag{2}$$

Proof. We prove the claim by induction on n.

Base case ($n = 0$). The sum on the left-hand side of Equation (2) is empty, so it is $0 = 0^2$.

Induction step ($n \to n+1$). As induction hypothesis, assume that Equation (2) holds for n. By splitting off the last summand and applying the induction hypothesis, we obtain

$$\sum_{i=1}^{n+1}(2i-1) = \sum_{i=1}^{n}(2i-1) + 2(n+1) - 1 = n^2 + 2n + 1 = (n+1)^2,$$

so Equation (2) also holds for $n+1$. □

Language and Style

- **Use "we"** consistently, meaning the author and the reader ("We now show that ..."). Never use "I"!
- **Prefer the active voice and the present tense** ("Theorem 3.2.2 shows ..."). Use the past tense only for things that happened at a specific time, e.g., when you describe the experiments you ran.
- **Be precise.** Replace vague words like "very", "quite", "some", or "a lot" by concrete statements or numbers ("solves 23 more examples" instead of "solves many more examples"). Avoid ambiguous "it" and "this" and name the object instead.
- **Abbreviations** are spelled out at their first use, e.g., "dependency pairs (DPs)", and are not used in the abstract.
- **Spelling.** Use either American or British English consistently, and run a spell checker before submission. Write "e.g.," and "i.e.," with a comma afterwards.

Figures and Tables

- **Every figure and table is referenced** in the text with `\cref`, and the text explains what the reader should see in it.
- **Captions are self-contained**: a reader who only looks at the figure and its caption should understand what is shown, including the meaning of all abbreviations and units.

- **Tables** use horizontal rules only (`\toprule`, `\midrule`, `\bottomrule` from `booktabs`) and no vertical rules, see Table 1 and Table 4.1.
- **Colors** must not be the only way to distinguish information: the thesis has to be readable in black and white and by readers with color vision deficiency.